Results and limits

What goal-orientation buys at equal compute, and where it does not help

Carlos Vera-Ciro

Goal-Oriented DFR series | Part 4

Where we are

- Part 2 built the method; Part 3 gave a computable bound on the QoI error.
- This deck asks the empirical question: at equal cost, does the QoI- weighted loss actually give a more accurate quantity of interest?
- The answer is yes, modestly, and only in one of the two settings we test. We report it honestly.

Section 1

What the method does

Goal-orientation reallocates accuracy

On a fixed problem, at the same budget, goal-oriented DFR piles the network's accuracy onto the QoI, and spends it there:

plain DFR: uniform *goal-oriented: piled at x_0*

- It does not solve the problem *better*; it solves a *different* problem, buying local accuracy with global: the global error grows to 9.8%, against 0.9% for plain DFR.
- **Warning for use:** a goal-oriented solution is trustworthy at its quantity of interest and nowhere else.

Takeaway: The mechanism is a trade, not a free lunch: local accuracy bought with global accuracy.

Section 2

Measuring the advantage

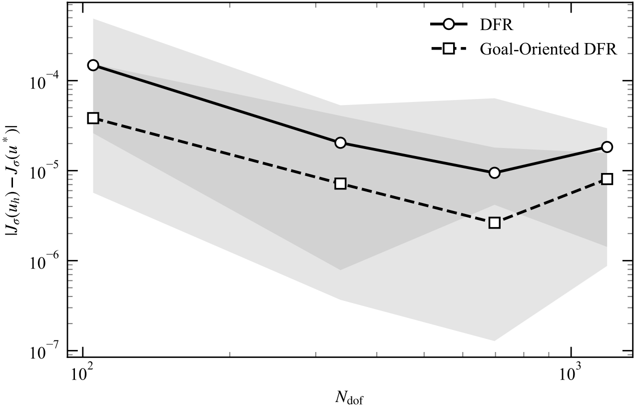
The comparison that matters: equal cost, not equal steps

Goal-orientation trains a **second network**, so it costs roughly 2× per step.

- Comparing at the same *step count* (matched budget) hands goal-orientation extra compute. It over-credits the method.
- The fair test gives plain DFR 2.5× the training, so both use the same **wall-clock** (matched cost).
- A second wrinkle: plain DFR is *not converged* at these budgets; its error keeps falling with more steps.

Takeaway: Read every headline twice: once at matched budget (an upper bound), once at matched cost (the honest measure).

Matched budget: the advantage looks large



2D Poisson, point QoI, sweeping the network size:

- Goal-orientation wins **33 of 40** runs.
- Median QoI error **2.3 to 3.9** times smaller (geometric mean 3.1×).
- But this is matched *budget*. Keep it as an upper bound.

Matched cost: the advantage roughly halves, and splits

Two-dimensional sweep: survives.

- Paired median $2.2\times$, 95% CI $[1.2, 4.6]$.
- Wins **28 of 40** runs. Clear of unity.

Steepness sweep: does not survive.

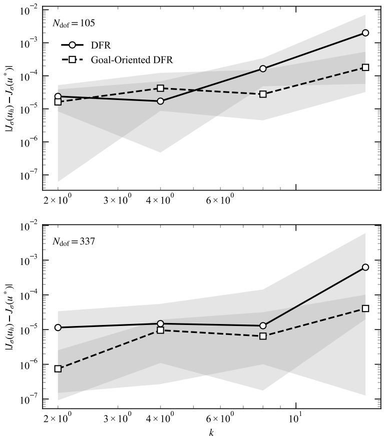
- Paired median $0.92\times$, CI $[0.69, 1.85]$.
- Wins **38 of 80**. Indistinguishable from parity.

	matched budget	matched cost
2D (size sweep)	$3.1\times$	$\mathbf{2.2}\times$
steepness	up to $15\times$	$\sim\mathbf{1}\times$

Geometric mean of the QoI-error improvement. At equal cost the steepness advantage vanishes into the seed noise.

Takeaway: The honest headline: a modest, setting-dependent $\sim 2\times$ gain at equal compute, not an order of magnitude.

Why the steepness advantage collapses



- At matched budget, plain DFR degrades sharply as the solution sharpens; goal-orientation less so.
- We had read this as "the network cannot represent the sharp solution."
- At matched cost it is mostly **under-convergence**: give plain DFR the extra budget and it catches up.

Section 3

Limits and takeaways

What we are not claiming

- Plain DFR is **not trained to convergence** even at 2.5$\times$ the budget; a fully converged comparison is out of reach at practical budgets.
- The method **can hurt**: on easy problems at small networks it is worse than the baseline.
- Goal-oriented solutions are $\sim$10$\times$ worse **globally**. Use them only where the QoI is all you need.
- A single problem family (arctan bumps), a mollified point QoI, box domains. Other settings untested.
- The theory predicts **none** of the observed gain. Explaining it is the open problem this work leaves.

The bottom line

A carefully delimited contribution

- A **method**: goal-oriented DFR, primal plus adjoint, with the stop-gradient that makes it work.
- A **theorem**: a computable, one-sided reliability bound on the QoI error, verified on 168 runs.
- A **finding**: the loss reallocates accuracy toward the QoI, and at equal compute this buys a modest $\sim 2\times$ in one setting and nothing reliable in another.

A real mechanism, an honest size, and a clear map of where it helps and where it does not.

Reproduce it

Everything is in the repository, one command per figure:

- `uv run hp-dfr data all` then `uv run hp-dfr figures` for the matched-budget sweeps and the bound verification.
- `uv run hp-dfr data matched-cost` for the equal-cost comparison (shardable; runs on the ECS infrastructure in `packages/infra`).
- Manuscript, code and docs: github.com/caverac/hp-dfr, caverac.github.io/hp-dfr.

Thank you.